

PathWitness

Removing redundant branches that `clang -O3` keeps

Team DietCode

SegFault 2026

September 17, 2026

SQLite stores a value's properties as bits of a 16-bit flags field.

```
if ((flags & 0x12) == 0) /* are bits 1 and 4 both clear? */  
    if ((flags & 0x202) == 0x202) /* are bits 1 and 9 both set? */
```

Bit 1 appears in both masks. If the first test passed, bit 1 is clear, so the second test cannot pass. The inner branch is unreachable.

Removing unreachable code is ordinary optimiser work. GCC does it here.

Background: how LLVM remembers what a branch proved

When LLVM takes a branch it records what that branch established, so later code can use it.

It files that fact under *the value the condition mentioned*. In `DomConditionCache` the store is a map from a value to the branches constraining it, and lookups are by that value.

The consequence we exploit

If a later test asks about a *different* value, even one that holds the same number, the lookup finds nothing and the fact is not used.

C integer promotion widens anything narrower than `int` before arithmetic. A 16-bit `flags` field becomes a 32-bit copy.

So one test may be written about the widened copy and another about the original. Two values, same number, different identity.

This is not exotic: it happens around every `unsigned short` or `uint8_t` field, and inlining creates more of it.

The issue: a test that can never be true, kept at -03

sqlite3_value_blob, from unmodified SQLite 3.45.1:

```
%3 = load i16, ptr %2 ; the flags field, 16 bits wide
%4 = zext i16 %3 to i32 ; the widened copy
%5 = and i32 %4, 18 ; guard tests the WIDE copy (0x12)
%6 = icmp eq i32 %5, 0
br i1 %6, label %44, label %7 ; reaching %44 proves bit 1 is CLEAR
```

44:

```
%45 = and i16 %3, 514 ; query tests the NARROW original (0x202)
%46 = icmp eq i16 %45, 514 ; requires bit 1 SET
br i1 %46, label %47, label %54
```

The guard's fact is filed under %4. The query asks about %3. The branch, and two basic blocks behind it, survive -03.

The issue is real, not a toy

- GCC 14, given the same file, emits no such test. In the assembly of this one function clang has `testl $514` and gcc has nothing.
- It does *not* reduce to a small example. Write the same pattern in ten lines of C and clang folds it. The miss needs the surrounding function.

Why the existing fix does not fire

InstCombine can already narrow such a test, but only when the widened value has exactly one consumer. `tests/one-use-boundary.ll` holds the same function twice, differing by one extra use: `stock opt -passes=instcombine` narrows the first and leaves the second alone.

Our solution: two passes that only restate facts

Neither pass deletes a branch or claims anything is unreachable. Each removes the mismatch and lets LLVM's own cleanup act.

1. Mask width. Zero-extension leaves the high bits clear, so testing them is redundant:

$$((\text{zext } X) \& M) == C \longrightarrow (X \& \text{trunc}(M)) == \text{trunc}(C)$$

We decide for *all* consumers of a widened value at once. If every one can be narrowed we narrow them together and the widening dies. If even one cannot, we change nothing.

2. Shift mask. For $(1 \ll (x \& C)) \& M$, a branch says which values $x \& C$ can still take. We enumerate them, which is bounded and exact, and state the bits that are fixed as an `llvm.assume`.

We did not want to assert the problem, so we measure it.

- Every branch edge goes to the Z3 solver with the facts carried by the edges that dominate it. It counts only if no input can reach it.
- Every finding is then re-checked by a second tool that uses no solver, walking the IR with concrete inputs to try to reach the branch.
- `before` and `after` compile the same sources with the same flags and the same cleanup. The only difference is whether our two passes ran first.

Eight libraries that ship in production, pinned by checksum: SQLite, zlib, Lua, cJSON, bzip2, lz4, miniz, zip.

Functions analysed	3,034
Branch edges queried	44,572
Edges no input can reach, kept by -03	37

Five of the eight libraries contain none at all. We report that as a finding about how rare the pattern is, not as a failure.

The count is a lower bound: unmodelled operations become free variables and timed-out queries are discarded.

Result: what our passes remove

	Before	After	Removed
Unreachable branches kept	37	29	8

Not all 37 are ours to fix. 12 need bitmask reasoning *across a width cast*, and that is the only thing these passes address. 8 of those 12 are gone, in 6 functions.

Removed by the mask-width pass alone	3
Removed by the shift-mask pass alone	5

The two sets are disjoint and the totals add, so the passes are independent: neither enables the other and neither is redundant.

Result: the function we started from

sqlite3_value_blob, on LLVM 23.1.1 with both passes:

	before	after
basic blocks	14	11
instructions	63	52
conditional branches	9	7
machine code	150 bytes	125 bytes

Across the whole SQLite module the passes fire 545 times: 14 functions shrink, 7 grow, a net saving of 244 bytes. Functions grow because inlining decisions shift once a branch disappears. We report both directions.

Does it still work?

- Real SQLite compiled with the passes answers its API checks byte-identically: 15 SQL groups, 1,044 calls into the changed function.
- 37 scope controls: each shape we accept fires exactly once, each shape we reject fires not at all. 5.8 M native executions.
- Every transformation runs under the LLVM verifier.
- Alive2 accepted 14 of our 16 mask-width submissions, correctly rejected a deliberately wrong mask, and was inconclusive on one.
- The shift-mask pass emits `llvm.assume`, where a wrong derivation is a miscompile rather than a missed optimisation. It has no Alive2 coverage, so every claim it makes is checked against concrete arithmetic over all 65,536 inputs.

This is not a speedup

The branches removed were never taken, so no work disappears at run time. We measured no reliable timing difference and would not expect one.

- Our own detector produced one false positive. The independent re-checker caught it and we exclude it from every number here.
- Machine code does not always shrink; some functions grow.
- The bit identity is not new and we do not claim it. PR #132124 and PR #221189 are the closest upstream work; both are still restricted to a single consumer, which is the restriction our rule avoids.
- Eight libraries is not a survey of all C code.

Reproduce it, in four commands

```
pip install -r requirements.txt
```

```
python3 run.py setup # fetch, build, check
```

```
python3 run.py before # -> results/BEFORE.md
```

```
python3 run.py after # -> results/AFTER.md
```

```
python3 run.py compare # -> results/COMPARISON.md
```

The two reports share a structure, so `diff results/BEFORE.md results/AFTER.md` shows exactly what changed.